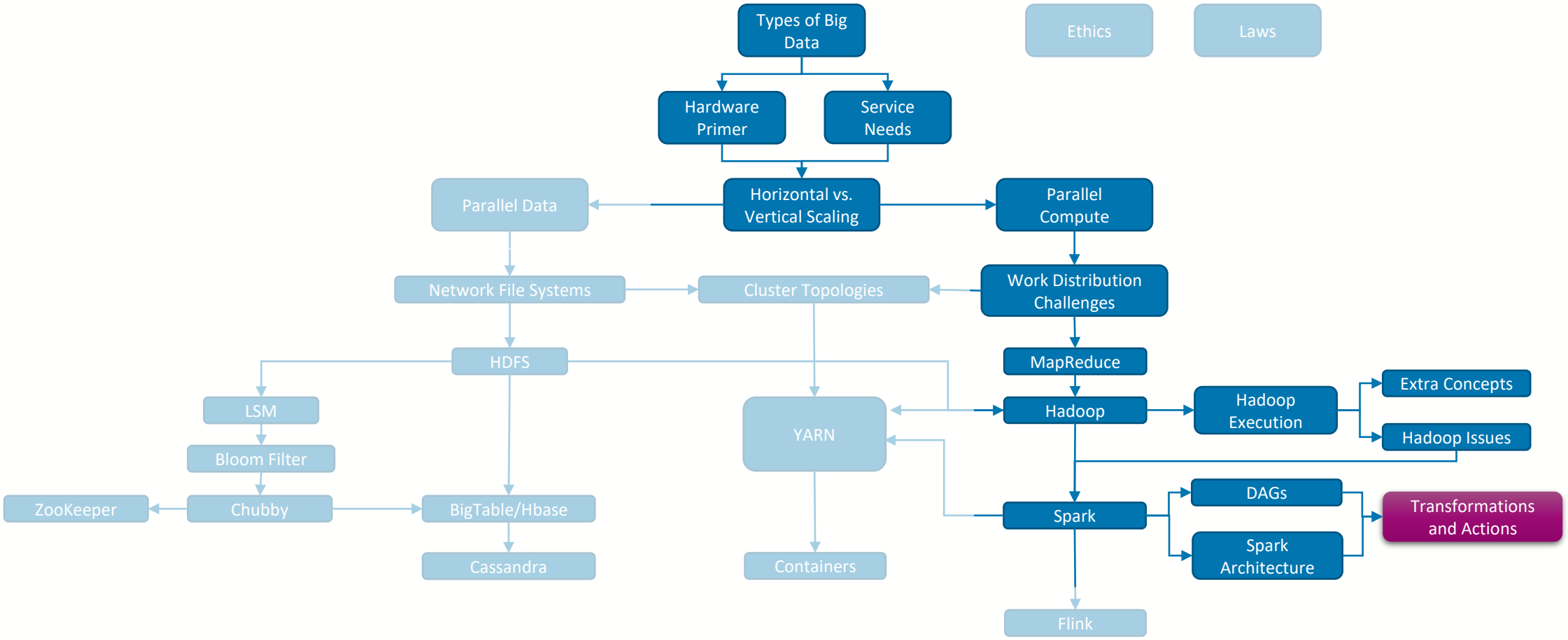
W4-02: Spark Basic Transformations

COMPSCI4064 (H) and COMPSCI5088 (M)

Dr. Richard McCreadie

University of Glasgow

Where are we?



University of Glasgow

Transformations and Actions

- Tasks in Spark are categorized into two main types
  - **Transformations**: These convert one (or more) RDD into another RDD

  - **Actions**: These either return an RDD (or metadata about an RDD) to the user's machine or writes the RDD to persistent storage
    - E.g. Collect converts an RDD to a Java List

Current Spark Transformations and Actions

Transformations

Basic
- map(func)
- filter(func)
- flatMap(func)
- mapPartitions(func)
- mapPartitionsWithIndex(func)
- sample(withReplacement, fraction, seed)

Set
- union(otherDataset)
- intersection(otherDataset)
- distinct([numPartitions]))

Key/Value
- groupByKey([numPartitions])
- reduceByKey(func, [numPartitions])
- aggregateByKey(zeroValue)(seqOp, combOp, [numPartitions])
- sortByKey([ascending], [numPartitions])

Set Key/Value
- join(*otherDataset*, [*numPartitions*])
- cogroup(*otherDataset*, [*numPartitions*])
- cartesian(otherDataset)

- pipe(command, [envVars])

Re-Partition
- coalesce(numPartitions)
- repartition(numPartitions)
- repartitionAndSortWithinPartitions(partitioner)

Actions

- reduce(func)
- collect()
- count()

Sample
- first()
- take(n)
- takeSample(withReplacement, num, [seed])
- takeOrdered(n, [ordering])

Save
- saveAsTextFile(path)
- saveAsSequenceFile(path)
- saveAsObjectFile(path)

- countByKey()
- foreach(func)

(User-Defined) Functions

Alteration Building Blocks

University of Glasgow

Custom Functions

- At the core of many transformations and actions is a transformation function
- This is simply a Java or Scala class of a pre-defined type, where that type specifies what the inputs and outputs of the function are, e.g.
  - Map Function: record1 ➡ record2
  - Reduce Function: record1,record1 ➡ record1
  - FlatMap Function: record1 ➡ List<record2>

- Transformations and Actions often take one of these functions as input, and then apply it to the data
  - Its easy to think of the function and the transformation as the same thing, but this is not the case, indeed the same function may be used in different tasks, such as reduce being used in both the reduce action and the reduceByKey transformation

University of Glasgow

Custom Functions

- What you have been working on in the Labs and will be doing for your assessed exercise is writing custom functions in Java to use in your Spark programs

- Spark can directly use functions written in either Java or Scala, so long as they extend the appropriate interface, e.g. MapFunction
  - If you use a third-party framework like Databricks or a wrapper like pySpark, these are converting the instructions you provide into the underlying Spark program using built-in Spark functions

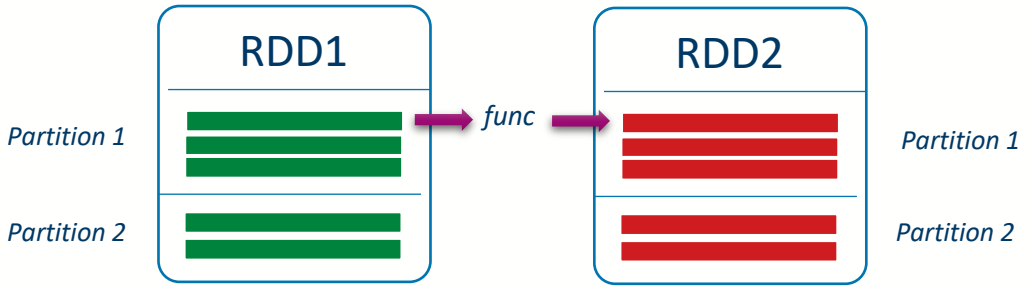
Basic Transformations

Converting one RDD to another

University of Glasgow

map(*func*)

- Description: Return a new distributed dataset formed by passing each element of the source through a function *func*.
- Function: record1 ➡ record2 MapFunction
- Application: Applied to each record in the RDD. Records in different partitions can be processed in parallel.



University of Glasgow

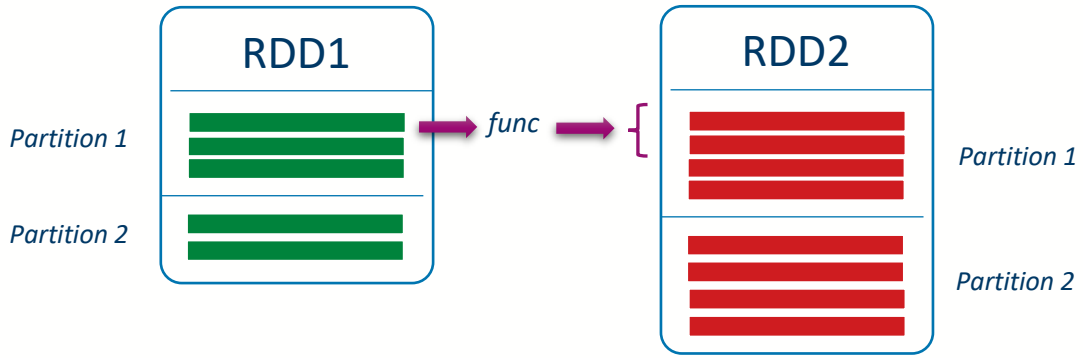
flatMap(func)

- Description: Similar to map, but each input item can be mapped to 0 or more output items (so *func* should return a Seq rather than a single item).
- Function: record1 ➡ List<record2>

FlatMapFunction

Iterator<record2>

- Application: Records can be processed in parallel. The output list from the function can contain 0 or more records.



University of Glasgow

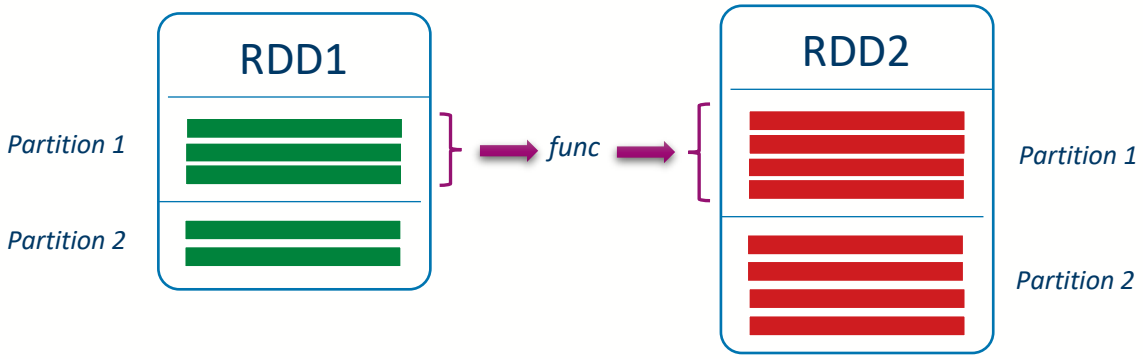
mapPartitions(*func*)

- Description: Similar to map, but runs separately on each partition (block) of the RDD, so *func* must be of type Iterator<T> => Iterator<U> when running on an RDD of type T.
- Function: List<record1> ➡ List<record2>

A Partition

Iterator<record2>

- Application: Partitions are processed in parallel. The input partition must contain at least one record. The output list from the function can contain 0 or more records.



University of Glasgow

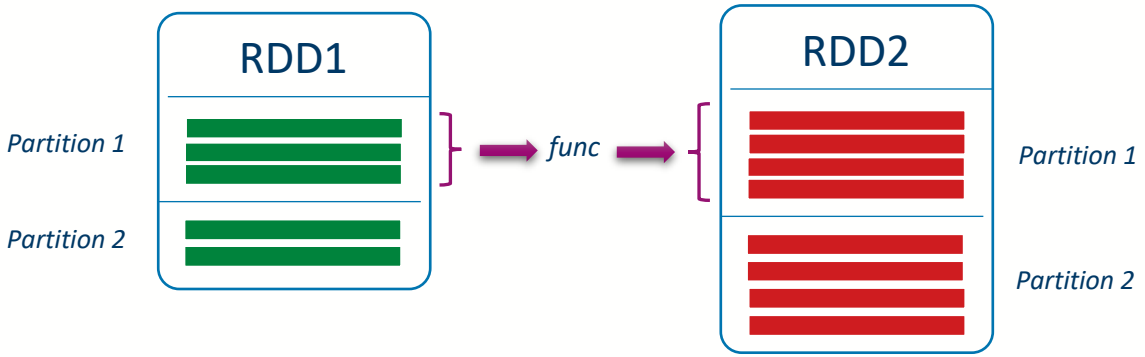
mapPartitionsWithIndex(*func*)

- Description: Similar to mapPartitions, but also provides *func* with an integer value representing the index of the partition, so *func* must be of type (Int, Iterator<T>) => Iterator<U> when running on an RDD of type T.
- Function: <index,List<record1>> ➡ List<record2>

A Partition

Iterator<record2>

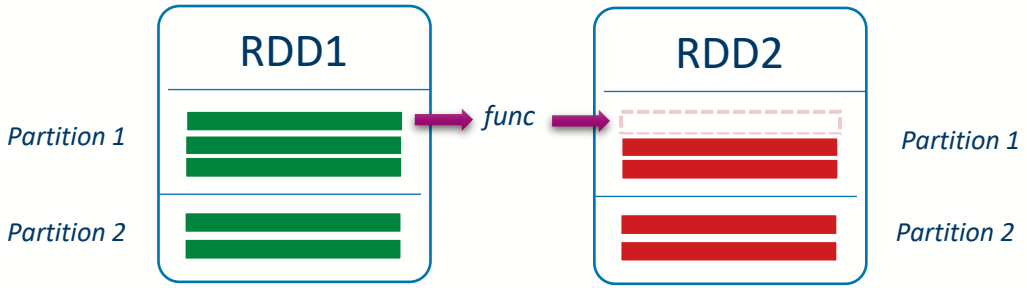
- Application: Partitions are processed in parallel. The input partition must contain at least one record. The output list from the function can contain 0 or more records. Useful if your partitions represent meaningful data subsets.



University of Glasgow

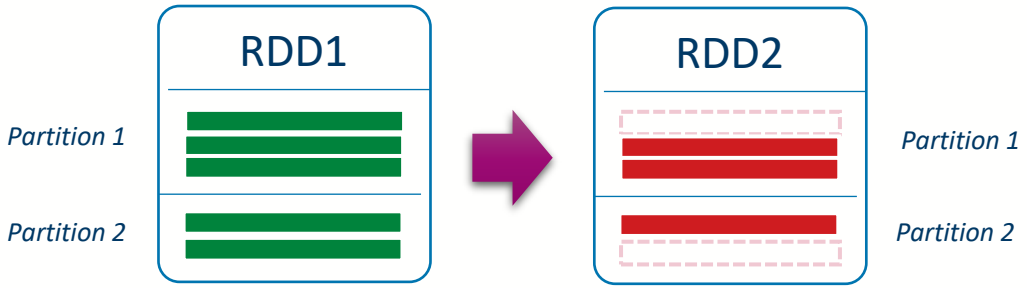
filter(*func*)

- Description: Return a new dataset formed by selecting those elements of the source on which *func* returns true.
- Function: record1 ➡ boolean MapFunction
- Application: Applied to each record in the RDD. If the function returns true the record is maintained in the output RDD, otherwise the record is dropped. Records in different partitions can be processed in parallel.



sample(*withReplacement*, *fraction*, *seed*)

- Description: Sample a fraction *fraction* of the data, with or without replacement, using a given random number generator seed.
- Function: Built-in
- Application: This is in effect a random filter function, where you can specify the proportion of the data you want to keep.



Course Coordinator
Dr. Richard McCreadie

Email: richard.mccreadie@glasgow.ac.uk
Room: SAWB 304

#UofGWorldChangers
@UofGlasgow